

Report for exercise 1 from group 27

Tasks addressed: 4
Authors: Moritz Beste (03767020)
Katharina Miller (03745313)
Mykhailo Shtandenko (03816036)
Last compiled: 2026-05-13

Report on task 1, Monte Carlo sampling estimator

Task 1.1 For this task the mean and variance is computed for the grades vector G given in `G.txt`. To numerically compute the mean and variance, the formulas Equation 1 and Equation 2 given in the assignment were used respectively:

$$\mathbb{E}[x] \approx \bar{X} = \frac{1}{N} \sum_{i=1}^N X_i \quad (1)$$

and

$$\mathbb{V}[X] \approx \hat{\sigma}_X^2 = \frac{1}{N-1} \sum_{i=1}^N (X_i - \bar{X})^2 \quad (2)$$

Where X_i are independent and identically distributed random variables, with $i \in \{1, \dots, N\}$. It is important to note that $\mathbb{V}[X]$ is the **Sample Variance** and not the **Population Variance**.

```

1 def python_compute(array: npt.NDArray) -> tuple[float, float]:
2     # =====
3     mean = 0
4     for grade in array:
5         mean += grade
6     mean /= len(array)
7
8     var = 0
9     for grade in array:
10        var += (grade - mean)**2
11    var /= (len(array) - 1)
12    # =====
13    return mean, var

```

Listing 1: Mean and Variance Computation Using Plain Python

```

1 def numpy_compute(array: npt.NDArray, ddof: int = 0) -> tuple[float, float]:
2     # =====
3     mean = np.mean(array)
4     var = np.var(array, ddof=ddof)
5     # =====
6     return mean, var

```

Listing 2: Mean and Variance Computation Using Numpy

These formulas were implemented using plain python Listing 1 and compared to the results computed using numpy's functions `mean()` and `var()` Listing 2.

It is important to note that the default value for parameter `ddof` in `numpy_compute()` is defined as 0. However, a value of 0 would correspond to computing the **Population Variance**. In order to compute the **Sample Variance**, a `ddof` value of 1 should be passed.

The mean and variance computed for the grades vector G using plain python and numpy were compared using Listing 3.

```

1 # =====
2 arr = load_grades(filename="./data/G.txt")
3 mean_py, var_py = python_compute(arr)
4 mean_np, var_np = numpy_compute(arr, ddof=1)
5
6 print(f"\nMean Computation\nPython Result: {mean_py}\nNumpy Result: {mean_np}\nResults Equal?
7     {mean_py == mean_np}")
8 print(f"\nVariance Computation\nPython Result: {var_py}\nNumpy Result: {var_np}\nResults
9     Equal? {var_py == var_np}")
10 # =====

```

Listing 3: Mean and Variance Comparison between Plain Python and Numpy

The computed results did match exactly. If it had come to small deviations, these deviations could have been due to algorithmic optimizations implemented in numpy and numeric instability. However for the grade vector G no deviations occurred.

Task 1.2 In this task, ranges of samples of length N with $N \in \{10, 100, 1000, 10000\}$ were generated using a predefined mean target $\mu = [-0.4, 1.1]^T$ and covariance target of $V = [[2, 0.4], [0.4, 1]]$. Then, these samples were used to numerically approximate the mean vector $\bar{X} \stackrel{!}{\approx} \mu$ and covariance matrix $cov \stackrel{!}{\approx} V$ using the given formulas Equation 3 and Equation 4 respectively.

$$\bar{X}_j = \frac{1}{N} \sum_{n=1}^N X_{j,n} \quad (3)$$

and

$$cov_{i,j} = \frac{1}{N-1} \sum_{n=1}^N (X_{i,n} - \bar{X}_i)(X_{j,n} - \bar{X}_j) \quad (4)$$

This was accomplished using a series of steps. First, the parameters were defined in code Listing 4.

```
1 mu = np.array([-0.4, 1.1])
2 V = np.array([[2, 0.4], [0.4, 1]])
3 sample_sizes = [10, 100, 1000, 10000]
```

Listing 4: Assignment 1.2 Parameters

For each sample size $N \in \{10, 100, 1000, 10000\}$, samples were created using numpy's `random.multivariate_normal()` function. This function allows for the painless generation of random samples using a normal distribution in appropriate dimensions given a mean vector, covariance matrix, and number of samples Listing 5.

```
1 def sample_normal(n_samples: int, mu_target: npt.NDArray, V_target: npt.NDArray, seed: int =
2     42) -> npt.NDArray:
3     # =====
4     np.random.seed(seed)
5     samples = np.random.multivariate_normal(mu_target, V_target, size=n_samples).T
6     # =====
7     return samples
```

Listing 5: sample_normal()

The next step is to approximate the mean vector $\bar{X}$ and covariance matrix cov . For this purpose, the `compute_moments()` function was implemented. It is important to note that numpy provides methods for the desired functionality. However, since the formulas (Equation 3 and Equation 4) were given in the assignment, these values were computed using plain python and compared to results computed by numpy to check for correctness. Since results may vary due to algorithmic deviations and numerical stability (especially due to the reliance on division), results were compared using an error tolerance of $\epsilon = 0.0001$.

```
1 def compute_moments(samples: npt.NDArray) -> tuple[npt.NDArray, npt.NDArray]:
2     # =====
3     np_mean = np.mean(samples, axis=1)
4     np_covariances = np.cov(samples, ddof=1)
5
6     py_mean = np.zeros(samples.shape[0])
7     for i in range(samples.shape[1]):
8         py_mean += np.array([samples[0][i], samples[1][i]])
9     py_mean /= samples.shape[1]
10
11     py_covariances = np.zeros((samples.shape[0], samples.shape[0]))
12     for n in range(samples.shape[1]):
13         add = [[0.0, 0.0], [0.0, 0.0]]
14         for i in range(samples.shape[0]):
15             for j in range(samples.shape[0]):
```

```

16         add[i][j] = (samples[i][n] - py_mean[i]) * (samples[j][n] - py_mean[j])
17         py_covariances += add
18         py_covariances /= samples.shape[1] - 1
19
20     eps = 0.0001
21     assert(np.isclose(np_mean, py_mean, atol=eps).all())
22     assert(np.isclose(np_covariances, py_covariances, atol=eps).all())
23     # =====
24     return py_mean, py_covariances

```

Listing 6: compute_moments()

Since the target mean μ and target covariance matrix V are known, the approximated results were compared to the target values. Specifically, the absolute mean error $|\bar{X} - \mu|$, the absolute diagonal covariance error (in one dimension) $|cov_{0,0} - V_{0,0}|$, the absolute off-diagonal covariance error (in one dimension) $|cov_{0,1} - V_{0,1}|$, and the RMSE $\hat{\sigma}_{\bar{X}} = \frac{\hat{\sigma}_X}{\sqrt{N}}$ were plotted Figure 1.

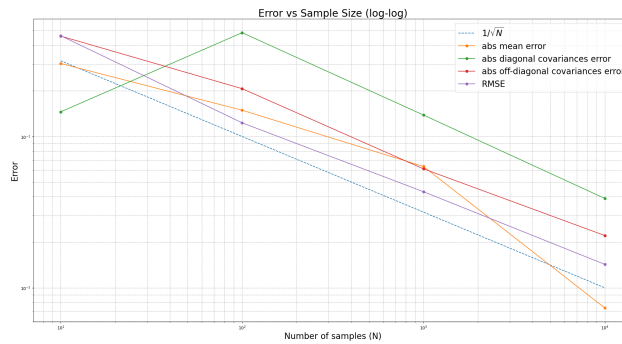


Figure 1: Estimation Errors for Various Number of Samples

Since this approximation is conducted using Monte-Carlo, the error should be proportional to $\frac{1}{\sqrt{N}}$, which is shown as a reference line in the graph Figure 1. This graph shows that the various plotted errors are relatively proportional to the reference line, which is expected. It is important to note though, that this graph shows only one run of the task, which means that the graph is influenced by variance. A more informative approach would be to run the experiment multiple times and plot the averages of results.

The formula for RMSE $\hat{\sigma}_{\bar{X}} = \frac{\hat{\sigma}_X}{\sqrt{N}}$ for the mean estimator can be derived from the definition of $RMSE = \sqrt{\mathbb{E}[\hat{\theta} - \theta]}$ for the mean estimator, however this is not possible using covariance, which means that this formula cannot be used for the covariance estimator.

Report on task 2, Monte Carlo integration

For this task the Monte Carlo sampling

$$\hat{I} = \frac{1}{N} \sum_{i=1}^N f(U_i), \quad U_i \sim \mathcal{U}(a, b) \quad (5)$$

should be used to approximate integrals via

$$I = \int_a^b f(x) dx \quad (6)$$

. That's useful as for Monte Carlo no analytical solution is needed and the implementation is quite simple. For this task different samples $N = [10, 100, 1000, 10000]$ are used.

Task 2.1

For this sub-task the integral

$$\int_0^1 \sin(x) dx = 1 - \cos(1) \quad (7)$$

should be approximated.

To do that the Sampling $U_i \sim \mathcal{U}(0,1)$ was generated. Afterwards the analytical solution of the integral I , the absolute error $\epsilon = |I - \hat{I}|$ and the RMSE for each N were calculated.

The following plot shows the difference between the two errors for each N . The RMSE reflects the expected magnitude of the error, whereas the exact error corresponds to a specific realization of the random estimator. Therefore, deviations between the two curves are to be expected.

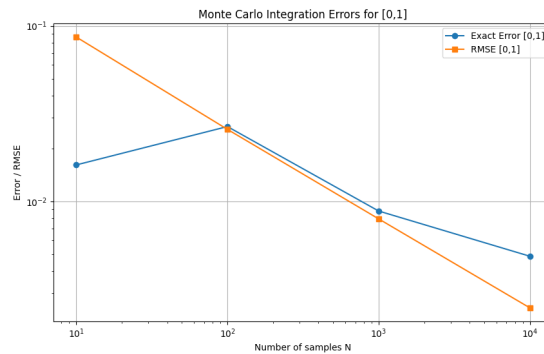


Figure 2: Comparison of absolute error and RMSE in the interval of $[0,1]$

In the log-log plot, the RMSE follows an approximately linear trend. The slope corresponds to a convergence rate of approximately $-\frac{1}{2}$, which confirms the theoretical Monte Carlo convergence behavior of $O(N^{-\frac{1}{2}})$. In comparison to that the absolute error is not linear at all. The non-monotonic behavior of the exact error (e.g., an increase from $N=10$ to $N=100$) is due to the stochastic nature of the estimator.

Task 2.2

For this sub-task the integral

$$\int_2^4 \sin(x) dx = -\cos(4) + \cos(2) \quad (8)$$

should be approximated. Thereby two different variants should be compared. The first kind of approximation is through direct approximation $X \sim \mathcal{U}(2,4)$. The second kind is through transformation. This was done by calculating $X = 2 + 2U$, $U \sim \mathcal{U}(0,1)$.

To do that for both variants the analytical solution of the integral I , the absolute error $\epsilon = |I - \hat{I}|$ and the RMSE for each N were calculated.

The following plot shows the difference of the both errors and variations.

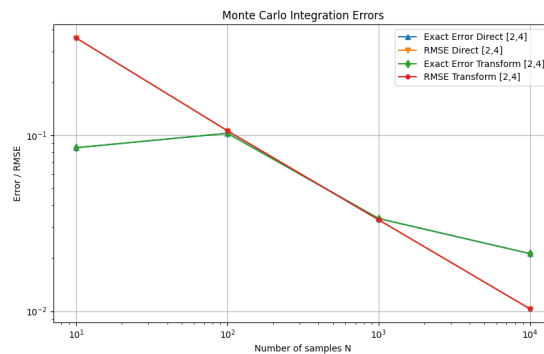


Figure 3: Comparison of absolute error and RMSE in the interval of $[2,4]$

It can be observed that sampling strategies yield identical results. This proves that it does not matter which strategy is used for approximation. Therefore, the RMSE has the same interpretation as in the previous sub-tasks, since both sampling strategies yield estimators with the same variance. The linear transformation has no effect on the convergence behavior.

Summary In summary, Monte Carlo integration is a simple and robust method for approximating integrals. The convergence rate is relatively slow ($O(N^{-\frac{1}{2}})$), but predictable. The RMSE is a reliable estimate of the expected error. Furthermore, different sampling strategies for uniform distributions yield equivalent results.

Report on task 3, Improving standard Monte Carlo sampling

To improve upon the baseline variance of standard Monte Carlo integration, we explore two advanced variance reduction techniques: the method of control variates and importance sampling. Our objective is to estimate the definite integral of the target function $f : [0, 1] \rightarrow \mathbb{R}$, defined as $f(x) = \exp(x)$, over the interval $[0, 1]$. To evaluate the convergence behavior and efficiency of each estimator, we conduct experiments across varying sample sizes, specifically $N \in \{10, 100, 1000, 10000\}$. As a baseline for comparison, we first calculate the absolute error of the naive Monte Carlo estimator for each value of N .

Next, we implement the method of control variates. This technique relies on introducing a highly correlated auxiliary function $\phi(x)$ with a known expected value to correct the primary estimator. The control variate estimator is defined as:

$$\hat{I}_{cv} = \bar{f} - c^*(\bar{\phi} - \mathbb{E}[\phi])$$

where:

- $\bar{f}$ is the sample mean of $f(X)$.
- $\bar{\phi}$ is the sample mean of $\phi(X)$.
- $\mathbb{E}[\phi]$ is the known `control_mean`.

To achieve optimal variance reduction, the coefficient c^* is dynamically computed using the empirical covariance of f and ϕ divided by the variance of ϕ :

$$c^* = \frac{\text{Cov}(f, \phi)}{\text{Var}(\phi)}$$

We selected three distinct polynomial functions for our control variates: $h_1(x) = x$, $h_2(x) = 1 + x$, and $h_3(x) = 1 + x + \frac{x^2}{2}$. Notably, these (last 2 functions) represent the successive Taylor series approximations of our target function e^x , theoretically providing increasing levels of correlation. The required variances and covariances are estimated using samples drawn from the standard uniform distribution $\mathcal{U}(0, 1)$. For each of the selected functions ϕ and for each sample size N , the absolute errors are computed.

Finally, we evaluate importance sampling, a technique that alters the sampling distribution to focus on regions of higher probability mass or functional magnitude. We employ the Beta distribution as our proposal distribution $q(x)$, comparing two distinct parameterizations: $\alpha = 5, \beta = 1$ (which skews sampling weight heavily toward the upper boundary) and $\alpha = 0.5, \beta = 0.5$ (which creates a U-shaped distribution). The importance sampling estimator is formulated as:

$$\hat{I}_{is} = \frac{1}{n} \sum_{i=1}^n f(x_i) \frac{p(x_i)}{q(x_i)}, \quad x_i \sim q$$

where $p(x)$ represents the target distribution and $q(x)$ represents the proposal distribution. The term $\frac{p(x_i)}{q(x_i)}$ serves as the importance weight to ensure the estimator remains unbiased.

The comparative results of all implemented methods are illustrated in the following figure:

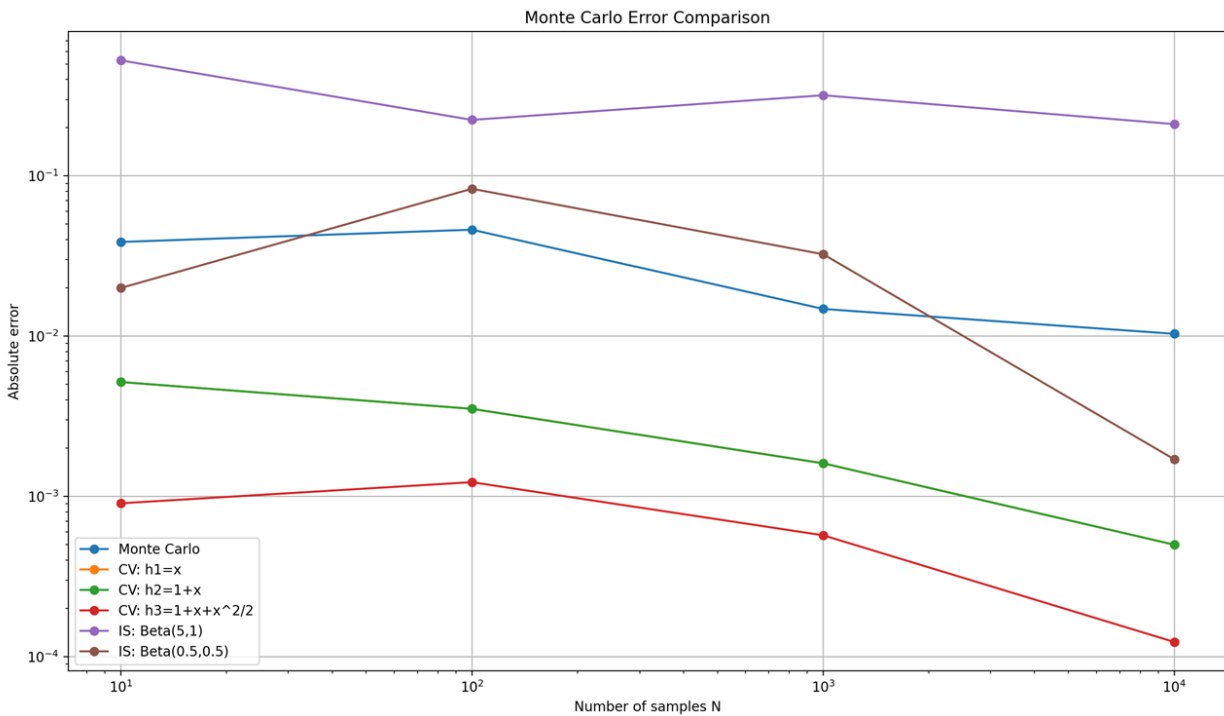


Figure 4: Comparison of absolute errors across different Monte Carlo variations.

Analyzing the log-log plot reveals several key insights. Notably, the error trajectories for the linear control variates $h_1(x) = x$ and $h_2(x) = 1 + x$ perfectly overlap (the orange and green lines). This occurs mathematically because adding a constant to a random variable does not alter its variance, nor does it change its covariance with another function. Consequently, the optimal coefficient c^* and the resulting variance reduction are completely identical for both linear approximations.

Ultimately, the most effective approach was the control variates method utilizing $h_3(x) = 1 + x + \frac{x^2}{2}$. Because this quadratic function acts as the second-order Taylor approximation of $\exp(x)$, it achieves the highest correlation with the target function, thereby minimizing the variance of the estimator far more effectively than the linear control variates or the selected importance sampling distributions.

Report on task 4, Monte Carlo sampling for forward propagation UQ

For this task, the aim was to propagate an error created through an uncertain function parameter through the program using Standard Monte Carlo and Quasi Monte Carlo using a Halton Sequence. The uncertain parameter was ω , which was modeled using the assumption that $\omega \sim \mathcal{U}(0.95, 1.05)$ and sampled with N numbers of samples where $N \in \{10, \dots, 10000\}$.

The first step for this task is to define the known hyperparameters as well as the `chaospy` distribution $\omega \sim \mathcal{U}(0.95, 1.05)$ for ω as defined in the problem statement.

It is important to note that the length of `t_grid` is 1001, which means that computing the trajectories for one omega takes 1001 iterations. Since the trajectories are computed for each sample of ω , this simulation is considerably expensive. Thankfully this problem can be classified as embarrassingly parallel, since the samples of ω can be distributed between different threads and results can be appended at the end of the simulation. For this purpose a worker task is defined, which is executed on each created thread (Listing 7).

```

1 def worker(model_kwargs: dict[str, float], init_cond: dict[str, float], t_grid: npt.NDArray,
2   omegas_chunk: npt.NDArray):
3     c = model_kwargs['c']
4     k = model_kwargs['k']
5     f = model_kwargs['f']
6     dt = model_kwargs['dt']

```

```

6
7     solutions_chunk = np.zeros((len(omegas_chunk), len(t_grid)))
8     for i, omega in enumerate(omegas_chunk):
9         y_t = init_cond['y_0']
10        dy_t = init_cond['y_1']
11        for j, t in enumerate(t_grid):
12            d2y_t = f * np.cos(omega * t) - c * dy_t - k * y_t
13            dy_t = dy_t + dt * d2y_t
14            y_t = y_t + dt * dy_t
15            solutions_chunk[i, j] = y_t
16
17     return solutions_chunk

```

Listing 7: Worker Task for Simulation

The worker threads are created and managed in the simulation function (Listing 8). Due to time constraints, the implementation for thread management is quite naive, as threads are forked and joined for each $n \in N$ and rule. A more sophisticated approach could be generating all ω chunks, then forking all threads, and using a work queue to pass each ω chunk to a worker. This would avoid forking and joining threads more than necessary, which is considerably expensive. However, the implemented naive approach already resulted in a speedup of approximately 5x on 16 hardware threads.

```

1  if rule.lower() == "random":
2      omegas = omega_distr.sample(size=n_samples, seed=seed)
3  elif rule.lower() == "halton":
4      halton_points = cp.create_halton_samples(n_samples, dim=1)
5      omegas = omega_distr.inv(halton_points).flatten()
6
7  sample_solutions = np.zeros((n_samples, len(t_grid)))
8  num_threads = min(n_samples, os.cpu_count() or 1)
9  omegas_chunks = np.array_split(omegas, num_threads)
10
11 # embarrassingly parallel problem
12 with ProcessPoolExecutor(max_workers=num_threads) as executor:
13     futures = []
14
15     # ensure correct placement of chunk in sample_solutions for reproducibility
16     start = 0
17     for chunk in omegas_chunks:
18         end = start + len(chunk)
19         future = executor.submit(worker, model_kwargs, init_cond, t_grid, chunk)
20         futures.append((future, start, end))
21         start = end
22
23     for future, start, end in futures:
24         sample_solutions[start:end] = future.result()

```

Listing 8: Generation of ω Samples and Distribution to Worker Threads

Next, the final results $y(10)$ are retrieved from the computed `sample_solutions` and used to compute a mean and variance between sampled ω s for each $n \in N$ and rule. To compute the mean and variance, we use formulas Equation 1 and Equation 2 respectively, although `numpy` functionality was used for simplicity. These values were used to compute a relative error using the given formula $rel_{error} = |1 - \frac{approx}{ref}|$ compared to reference values computed using $n = 10^6$ samples (Listing 9). Once again, the `sample variance` was computed by passing `ddof=1` to `numpy.var()`.

```

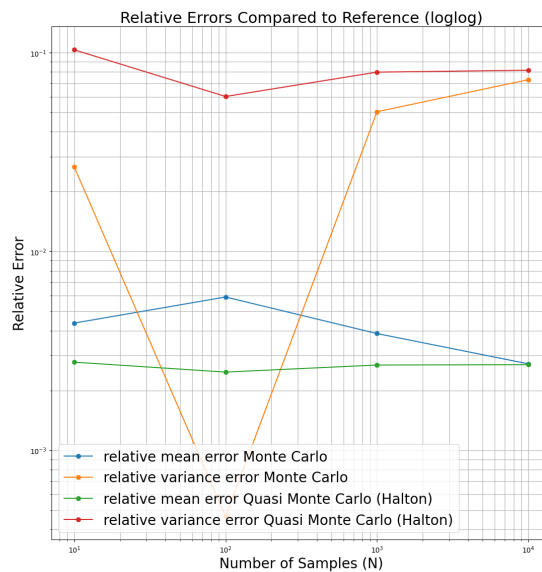
1  def compute_errors(
2      samples: npt.NDArray, mean_ref: float, var_ref: float
3  ) -> tuple[float, float]:
4      # =====
5      mean = np.mean(samples)
6      var = np.var(samples, ddof=1)
7
8      mean_error = abs(1 - mean / mean_ref)
9      var_error = abs(1 - var / var_ref)
10     # =====

```

```
11 | return mean_error, var_error
```

Listing 9: Computation of Relative error of Mean and Variance

Figure 5 shows the computed relative errors plotted on a log-log scale. This graph shows some interesting behavior. It is especially striking that there only seems to be weak performance improvements with higher number of samples. This may be due to the variance of $\omega \sim \mathcal{U}(0.95, 1.05)$ being relatively small at $Var(\omega) = \frac{(1.05-0.95)^2}{12} \approx 8.33 * 10^{-4}$. Furthermore, the relative error of the variance using Standard Monte Carlo is quite small at $n = 10^2$ samples, which may indicate that the sampled seed was lucky at $n = 10^2$. However, it is important to note that this graph only shows the experiment being conducted once. More comprehensive experiments with multiple different sampled seeds are required to gain better understanding of the underlying behavior.

Figure 5: Relative Error Compared to Reference using $N = \{10, \dots, 10000\}$

Furthermore, the trajectories of 10 different sampled ω s was graphed for $t \in [0, 10]$ with $\delta t = 0.01$ (Figure 6). This graph shows that the behavior starts to deviate more and more as time goes on with different ω . Especially for $t \in [0, 1]$ the graph shows very little divergence, which is expected with slightly varying frequency ω . However, even with just $n = 10$ samples, the final deviation between trajectories is relatively small, which corroborates the behavior of small relative errors shown in Figure 5.

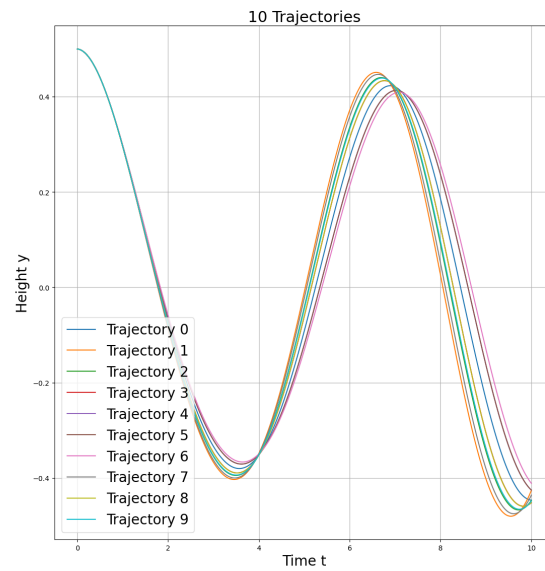


Figure 6: Trajectories of Oscillator using 10 Different Standard Monte Carlo Samples